

```

/*
* ULTIMATE B1S2 O2 SENSOR SIMULATOR - v3.3.1 (Rationality Dip Version)
* -----
* TID:$8b | Rich Test          | Min: 0.825V | Max: 7.995V | Result: 0.880V
* TID:$8c | Lean Test          | Min: 0.000V | Max: 0.100V | Result: 0.045V
* TID:$91 | Voltage Average    | Min: 0.000V | Max: 0.450V | Result: 0.760V*
* TID:$a0 | Catalyst Monitor    | Min: 0.349  | Max: 3.999  | Result: 0.760V
* -----
* FIX: Added 1-second dip to 0.450V every 70 seconds to satisfy P2097 logic.
*/

```

```

// --- PINS & GLOBAL VARIABLES ---

```

```

const int sensorIn = A1;
const int simOut = A0;

```

```

// Timer for P2097 Rationality Dip
unsigned long lastDipTime = 0;
const unsigned long DIP_INTERVAL = 70000; // 70 seconds
const unsigned long DIP_DURATION = 1000;  // 1 seconds

```

```

// Mode $06 Tuned Targets
const float TARGET_CRUISE_BASE = 0.795;
const float TARGET_RICH        = 0.890;
const float TARGET_LEAN        = 0.045;

```

```

// Safety Constraints
const float SAFE_AVG_MAX = 0.820;
const float SAFE_AVG_MIN = 0.780;

```

```

float currentOutput = 0.255;
float initialBias = 0.0;
float rollingAvgIn = 0.450;
bool programRunning = false;

```

```

int lowCounter = 0;
int highCounter = 0;

```

```

void setup() {
  pinMode(simOut, INPUT);
  analogReadResolution(12);
  analogWriteResolution(10);
  delay(100);
  initialBias = (analogRead(sensorIn) / 4095.0) * 3.3;
}

```

```

void loop() {
  int rawRead = analogRead(sensorIn);
  float voltageIn = (rawRead / 4095.0) * 3.3;

  if (!programRunning) {

```

```

    if (abs(voltageIn - initialBias) > 0.150) {
        pinMode(simOut, OUTPUT);
        programRunning = true;
    }
    return;
}

// 6. DETECTION THRESHOLDS
if (voltageIn < 0.08) {
    lowCounter++;
    highCounter = 0;
}
else if (voltageIn > 0.85) {
    highCounter++;
    lowCounter = 0;
}
else {
    if (lowCounter > 0) lowCounter--;
    if (highCounter > 0) highCounter--;
}

float target;
unsigned long currentTime = millis();

// 7. THE LOGIC BRAIN
if (lowCounter > 110) {
    target = TARGET_LEAN;
    currentOutput = (currentOutput * 0.15) + (target * 0.85);
}
else if (highCounter > 80) {
    target = TARGET_RICH;
    currentOutput = (currentOutput * 0.15) + (target * 0.85);
}
// --- START RATIONALITY DIP LOGIC ---
else if (currentTime - lastDipTime < DIP_DURATION) {
    // 1-second dip to roughly 0.450V to prevent P2097
    target = 0.400 + (random(0, 50) / 1000.0);
    currentOutput = (currentOutput * 0.70) + (target * 0.30);
}
else if (currentTime - lastDipTime > DIP_INTERVAL) {
    lastDipTime = currentTime;
    target = TARGET_CRUISE_BASE; // Placeholder for logic flow
}
// --- END RATIONALITY DIP LOGIC ---
else {
    rollingAvgIn = (rollingAvgIn * 0.998) + (voltageIn * 0.002);
    float dynamicBias = (rollingAvgIn - 0.450) * 0.015;

    float heartbeat = sin(millis() / 1500.0) * 0.020;

```

```

    target = TARGET_CRUISE_BASE + dynamicBias + heartbeat;

    if (target > SAFE_AVG_MAX) target = SAFE_AVG_MAX;
    if (target < SAFE_AVG_MIN) target = SAFE_AVG_MIN;

    if (abs(currentOutput - target) > 0.05) {
        currentOutput = (currentOutput * 0.80) + (target * 0.20);
    } else {
        currentOutput = (currentOutput * 0.950) + (target * 0.050);
    }
}

// 8. REALISM JITTER
float jitter = (random(0, 5) / 1000.0);

float finalVoltage = currentOutput + jitter;

if (finalVoltage > 0.900) finalVoltage = 0.900;
if (finalVoltage < 0.000) finalVoltage = 0.000;

int dacValue = (finalVoltage / 3.3) * 1023;
analogWrite(simOut, dacValue);

delay(25);
}

```